

27. live 計測鮮度監視

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/solar_preflight_node.py
種別	Python
区分	runtime node
役割	speed、distance、battery、planner status の鮮度を見て system/state と health を publish する。
起動文脈	起動可否と運用中の健全性の監視役。

このファイルを読む理由

speed、distance、battery、planner status の鮮度を見て system/state と health を publish する。

- planner 自体は止めずに、system/state と health を出す。

呼び出し関係

どこから呼ばれるか

- mpc_solarcar/live_launch.py
- launch/solar_measurement.launch.py

次に読むと理解が進むファイル

- mpc_solarcar/solar_preflight_logic.py
- mpc_solarcar/speed_command_bridge_node.py

同一リポジトリ内の主要依存

- mpc_solarcar/solar_preflight_logic.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
from .solar_preflight_logic import evaluate_freshness

class SolarPreflightNode(Node):
    """Preflight monitor based only on solar-car telemetry freshness."""

    def __init__(self):
        super().__init__("solar_preflight_node")
        self.declare_parameter("startup_grace_sec", 10.0)
        self.declare_parameter("measurement_timeout_sec", 3.0)
        self.declare_parameter("require_speed", True)
        self.declare_parameter("require_distance", True)
        self.declare_parameter("require_battery", True)
        self.declare_parameter("require_planner", True)

        self.start_mono = time.monotonic()
        self.last_seen: dict[str, float] = {}

        self.pub_state = self.create_publisher(String, "/system/state", 10)
        self.pub_health = self.create_publisher(Float32, "/system/health", 10)
        self.pub_diag = self.create_publisher(String, "/system/diag", 10)
```

主要構造の読み取り

主要クラスは SolarPreflightNode。主要関数は callback, main。ROS パラメータ宣言は 6 件。ROS I/O は publisher 3 件、subscription 6 件。

主要クラス・関数

行	種別	名前	補足
12	class	SolarPreflightNode	bases: Node
15	function	__init__	args: self
41	function	_seen	args: self, key
42	function	callback	args: _msg
47	function	_required	args: self
59	function	_step	args: self
78	function	main	

ファイルを上から読んだときの定義順

行	内容
12	クラス SolarPreflightNode を定義する。
78	関数 main を定義する。
88	条件 __name__ == '__main__' を判定し、真なら内部処理を行う。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>import time</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L24, L43, L60。
5	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L79, L82, L85。
6	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L12。
7	<code>from std_msgs. msg import Float32, Float32MultiArray, String</code>	Float32 や String など軽量メッセージで planner / vehicle 値を publish するため。このファイル内での主な使用位置は L27, L28, L29, L31, L32, L33, L34, L35, ...。
9	<code>from solar_preflight_ logic import evaluate_ freshness</code>	preflight 判定ロジックから evaluate_freshness を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/solar_preflight_logic.py。このファイル内での主な使用位置は L66。

関数・クラスを上から順に解説

L12 クラス SolarPreflightNode

- 定義: `SolarPreflightNode(bases=Node)`
- docstring: Preflight monitor based only on solar-car telemetry freshness.
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `String`, `__init__`, `_required`, `_seen`, `append`, `bool`, `create_publisher`, `create_subscription`, `create_timer`, `declare_parameter`, `evaluate_freshness`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。

1. docstring または説明文字列を置く。
2. 関数 `__init__` を定義する。
3. 関数 `_seen` を定義する。
4. 関数 `_required` を定義する。
5. 関数 `_step` を定義する。

L78 関数 main

- 定義: `main()`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `SolarPreflightNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。

1. `rclpy.init(...)` を実行する。
2. `node` に `SolarPreflightNode()` の結果を代入する。
3. 例外処理を伴う `try` ブロックを実行する。

設定値と入力パラメータ

行	名前	既定値・補足
17	startup_grace_sec	10.0
18	measurement_timeout_sec	3.0
19	require_speed	True
20	require_distance	True
21	require_battery	True
22	require_planner	True

ROS topic I/O

行	種別	topic	callback / 補足
27	publisher	/system/state	publisher
28	publisher	/system/health	publisher
29	publisher	/system/diag	publisher
31	subscription	/vehicle/speed_kmh	self._seen('speed')
32	subscription	/vehicle/s_km	self._seen('distance')
33	subscription	/vehicle/batt_soc	self._seen('batt_soc')
34	subscription	/vehicle/batt_voltage_v	self._seen('batt_voltage')
35	subscription	/vehicle/batt_current_a	self._seen('batt_current')
36	subscription	/planner/status	self._seen('planner')

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_preflight_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。